Problem A. Ask With Caution

Input file:	`standard input`
Output file:	`standard output`
Time limit:	4 seconds
Memory limit:	256 megabytes

This is an interactive problem.

Tasmi wants to play a game with you. She has chosen two hidden integers a and b ($1 \le a, b \le 1000$).

You may perform queries of one of the following two types:

- `MUL` k — replace a with $a \cdot k$; if the new value of a exceeds 10^9, then a becomes 0 instead,
- `DIV` k — if a is divisible by k, replace a with $\frac{a}{k}$; otherwise, a becomes 0.

After each query, Tasmi tells you the value of $\text{lcm}(a, b)$ — the least common multiple of a and b — for the updated value of a. Here $\text{lcm}(0, x) = 0$ for every x.

Your task is to determine the **initial** values of a and b using at most 10 queries.

Interaction Protocol

The first line of the input contains a single integer t ($1 \le t \le 2000$) — the number of games you must play. For each game, the interaction proceeds as follows.

To **perform a query**, output one line of the form `MUL` k or `DIV` k ($1 \le k \le 10^9$). After each query, read a single integer — the value of $\text{lcm}(a, b)$ for the updated value of a.

To **submit your answer**, output one line of the form `ANS` a b, where a and b are the **initial hidden integers**. Outputting the answer does not count as a query. After that, either proceed to the next game or terminate your program if this was the last game.

You can perform **at most** 10 queries per game; exceeding this limit, or outputting an invalid query, results in the `Wrong Answer` verdict.

All interaction keywords are case-sensitive. For example, you **cannot** use `"Div"`, `"div"`, or `"dIV"` in place of `"DIV"`.

After performing a query, do not forget to output the end of line and flush the output. Otherwise, you may get `Idleness Limit Exceeded` or `CPU Limit Exceeded` verdict. To do this, use:

- `fflush(stdout)` in C;
- `fflush(stdout)` or `std::cout.flush()` in C++;
- `System.out.flush()` in Java;
- `sys.stdout.flush()` in Python;
- `std::io::stdout().flush()` in Rust;
- Appropriate `flush` operation in any language.

Example

standard input	standard output
2 6 6 2 9682070000 0	 MUL 3 MUL 2 DIV 3 ANS 1 2 MUL 10000 DIV 991 ANS 977 991

Note

The interaction in the first sample game is shown in the following table. The first row shows the initial hidden integers. Each subsequent row shows the state of the hidden integers after the corresponding query and Tasmi's response, which is $\text{lcm}(a, b)$.

Query	a	b	Response	Explanation
	1	2		Initial value of the hidden integers
MUL 3	3	2	6	$a = 1 \cdot 3 = 3$, $\text{lcm}(3, 2) = 6$.
MUL 2	6	2	6	$a = 3 \cdot 2 = 6$, $\text{lcm}(6, 2) = 6$.
DIV 3	2	2	2	$a = \frac{6}{3} = 2$, $\text{lcm}(2, 2) = 2$.
ANS 1 2	2	2		Submitted answer matches the initial value of the hidden integers.

Note that the empty lines in the sample input and output are for the sake of clarity, and may not occur in the real interaction.

Problem B. Binary to Palindrome

Input file:	`standard input`
Output file:	`standard output`
Time limit:	1 second
Memory limit:	256 megabytes

You are given a binary string b of length n.

Construct a string s of length n consisting of lowercase English letters such that for each i ($1 \le i \le n$):

- if $b_i = 1$, then every substring* of s of length i is a palindrome†;
- if $b_i = 0$, then at least one substring of s of length i is not a palindrome.

Print the lexicographically smallest‡ such string s. If no such string exists, print -1.

* A string x is a substring of a string y if x can be obtained from y by the deletion of several (possibly, zero or all) characters from the beginning and several (possibly, zero or all) characters from the end. For example, `bc`, `abc`, and `d` are substrings of `abcd`, while `ac` and `ba` are not.

† A string is a palindrome if it reads the same from left to right and from right to left. In other words, the reverse of the string is the same as the original string. For example, `a`, `aba`, and `abba` are palindromes, while `ab` and `abc` are not.

‡ String x is lexicographically smaller than string y if either x is a proper prefix of y, or there exists a position i ($1 \le i \le \min(|x|, |y|)$) such that $x_i < y_i$ and for all $j < i$, $x_j = y_j$.

Input

The first line contains a single integer t ($1 \le t \le 10^4$) — the number of test cases.

The first line of each test case contains a single integer n ($1 \le n \le 3 \cdot 10^5$) — the length of the string b.

The second line contains a binary string b of length n consisting of the characters `0` and `1`.

It is guaranteed that the sum of n over all test cases does not exceed $3 \cdot 10^5$.

Output

For each test case, print the lexicographically smallest valid string s, or -1 if no such string exists.

Example

standard input	standard output
3 2 10 3 001 4 1111	ab -1 aaaa

Note

In the first test case, $b =$ `10`. Every substring of length 1 must be a palindrome, and the single substring of length 2 must not be a palindrome. The lexicographically smallest such string is `ab`.

In the second test case, $b =$ `001`. Since $b_1 = 0$ but a single character is always a palindrome, no valid string exists, so the answer is -1.

In the third test case, $b =$ `1111`. Every substring, of every length, must be a palindrome. The lexicographically smallest such string is `aaaa`.

Problem C. Stay in Your Lane

Input file:	`standard input`
Output file:	`standard output`
Time limit:	3 seconds
Memory limit:	512 megabytes

You are given an array $a_1, a_2, \ldots, a_n$ of n positive integers.

For a pair of indices (l, r) with $1 \le l \le r \le n$, consider the subarray $a_l, a_{l+1}, \ldots, a_r$ — the contiguous block of elements from index l to index r. This subarray is *self-contained* if every one of its elements lies in the range $[l, r]$; formally, if $l \le a_i \le r$ holds for every i with $l \le i \le r$.

Count the number of self-contained subarrays of a.

Input

The first line contains a single integer t ($1 \le t \le 10^5$) — the number of test cases.

The first line of each test case contains a single integer n ($1 \le n \le 10^6$) — the length of the array.

The second line contains n integers $a_1, a_2, \ldots, a_n$ ($1 \le a_i \le 10^9$) — the elements of the array.

It is guaranteed that the sum of n over all test cases does not exceed 10^6.

Output

For each test case, output a single integer — the number of self-contained subarrays of a.

Example

standard input	standard output
3 5 1 3 2 5 4 4 2 1 4 3 1 1	6 3 1

Note

In the first test case, $a = [1, 3, 2, 5, 4]$. The self-contained subarrays are:

- $(l, r) = (1, 1)$: the element 1 lies in $[1, 1]$;
- $(l, r) = (2, 3)$: the elements $3, 2$ lie in $[2, 3]$;
- $(l, r) = (4, 5)$: the elements $5, 4$ lie in $[4, 5]$;
- $(l, r) = (1, 3)$: the elements $1, 3, 2$ lie in $[1, 3]$;
- $(l, r) = (2, 5)$: the elements $3, 2, 5, 4$ lie in $[2, 5]$;
- $(l, r) = (1, 5)$: the elements $1, 3, 2, 5, 4$ lie in $[1, 5]$.

This gives 6 self-contained subarrays. As an example of a pair that does not qualify, $(l, r) = (2, 4)$ fails because $a_4 = 5$ does not lie in $[2, 4]$.

In the second test case, $a = [2, 1, 4, 3]$, the self-contained subarrays are $(1, 2)$, $(3, 4)$, and $(1, 4)$, for a total of 3.

In the third test case, $a = [1]$, and its only subarray $(1, 1)$ is self-contained since $a_1 = 1$, for a total of 1.

Problem D. Outsmarting

Input file:	`standard input`
Output file:	`standard output`
Time limit:	1 second
Memory limit:	256 megabytes

Alice and Bob play a game with strings.

Each game starts with Alice choosing a string A of length n consisting only of the characters '`H`' and '`T`' and revealing it to Bob.

The rest of the game is played according to the following procedure, in order:

- Bob chooses a string B of length n consisting only of the characters '`H`' and '`T`', such that $A \neq B$ and reveals the string to Alice.
- Now Alice and Bob build a new string S consisting only of the characters '`H`' and '`T`'. Bob chooses the first character of S.
- Then Alice keeps appending a new character ('`H`' or '`T`') to S until the game ends.

The game ends immediately when one of the following occurs:

- If A appears as a substring* of S, then Alice wins.
- If B appears as a substring of S, then Bob wins.
- If the length of S becomes 10^{100}, then Bob wins.

Both players know the rules of the game, and every action is immediately visible to both players.

Now, Alice has already chosen the string A and revealed it. Given n and A, determine the winner if both players play optimally from this point onward.

* A string a is a substring of a string b if a can be obtained from b by deleting several (possibly zero) characters from the beginning and several (possibly zero) characters from the end.

Input

The first line contains a single integer t ($1 \leq t \leq 10^4$) — the number of games. The description of the games follows.

The first line of each game contains a single integer n ($1 \leq n \leq 3 \cdot 10^5$) — the length of the strings.

The second line contains the string A of length n, consisting only of the characters '`H`' and '`T`' — Alice's chosen string.

It is guaranteed that the sum of n over all games does not exceed $3 \cdot 10^5$.

Output

For each game, output a single line containing `Alice` if Alice wins, and `Bob` otherwise.

Example

standard input	standard output
5 2 HT 3 THT 4 TTHT 3 HHH 1 T	Bob Alice Alice Bob Bob

Note

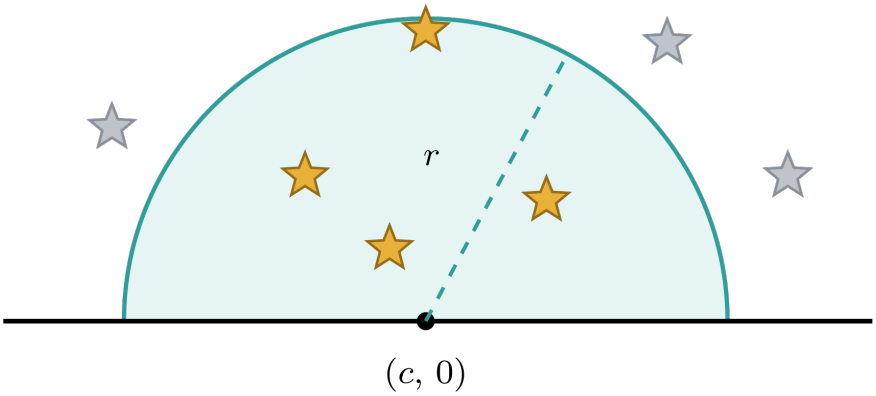
In the first game, $A =$ "HT". Bob can choose $B =$ "TH" and set the first character of S to 'T'. After that, no matter how Alice appends the remaining characters, Bob is guaranteed to win.

Problem E. Wonderful View

Input file:	`standard input`
Output file:	`standard output`
Time limit:	3 seconds
Memory limit:	512 megabytes

In Flatland, m stars lie above a straight horizon. Let the x-axis be the horizon. Then the i-th star is located at the point (x_i, y_i) where $y_i > 0$, and has beauty b_i.

You own a magic camera that stands on the horizon. Once initialized with a positive integer radius r, the camera can take a shot from any integer point c on the horizon, where $0 \le c \le n$: the shot captures a half-circle view of radius r centered at $(c, 0)$, and every star that lies at a distance of at most r from $(c, 0)$ is captured by it. The set of stars captured by a shot is called its *view*, and the *beauty* of a view is the sum of the beauties of the stars it contains.



The camera has a battery charge of k units, and every shot drains r units of it (recall that r is the radius of the circle), so you may take at most $\lfloor \frac{k}{r} \rfloor$ shots in total. The nice thing is, in the very beginning, you get to choose the value of r as any positive integer.

Shooting from the same point twice adds nothing the second time. Shooting from two different points always counts separately, however, even if the two shots happen to capture exactly the same stars.

Choose r and a set of distinct integer points to shoot from so that the total beauty of the views you obtain is maximized.

Input

Each test contains multiple test cases. The first line contains the number of test cases t ($1 \le t \le 100$). The description of the test cases follows.

The first line of each test case contains three integers n, k, m ($1 \le n \le 10^6$, $1 \le k \le 5 \cdot 10^6$, $1 \le m \le 10^3$) — the largest horizontal coordinate the camera can shoot from, the battery charge, and the number of stars.

Each of the next m lines contains three integers x_i, y_i, b_i ($0 \le x_i \le n$, $0 < y_i \le 10^6$, $1 \le b_i \le 10^9$) — the coordinates and beauty of the i-th star.

It is guaranteed that the sum of n over all test cases does not exceed 10^6, the sum of k over all test cases does not exceed $5 \cdot 10^6$, and the sum of m over all test cases does not exceed 10^3.

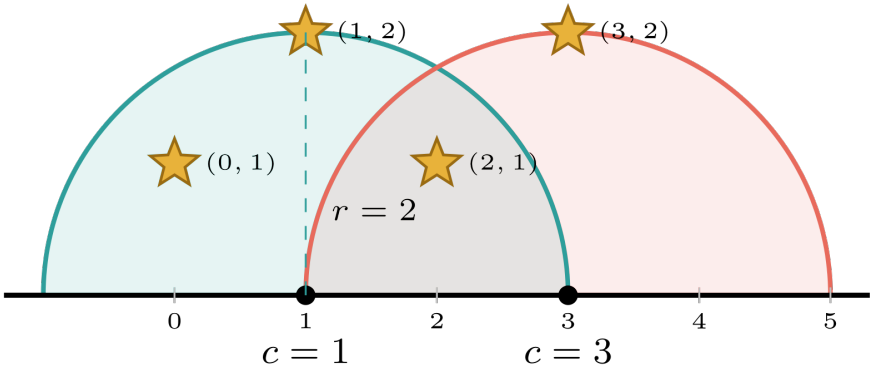
Output

For each test case, print a single integer — the maximum total beauty you can obtain.

Example

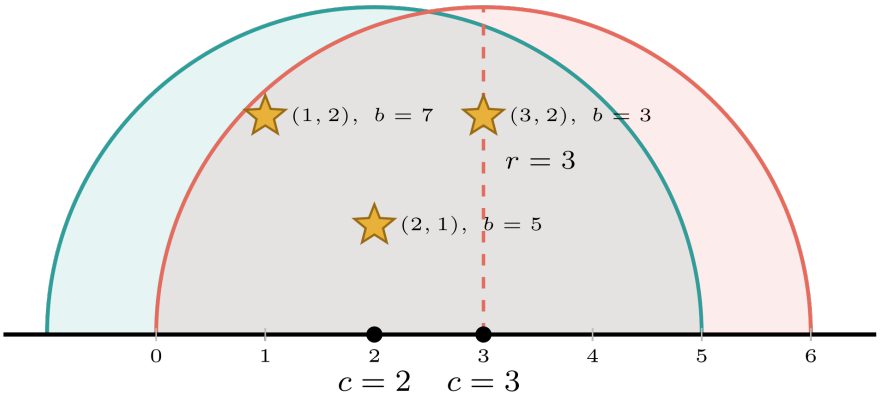
standard input	standard output
2 3 4 4 0 1 1 1 2 1 2 1 1 3 2 1 5 6 3 1 1 5 3 1 3 4 1 7	5 30

Note



In the first test case, one optimal choice is $r = 2$, which allows $\lfloor \frac{4}{2} \rfloor = 2$ shots since $k = 4$.

Shooting from $c = 1$ captures the stars at $(0, 1)$, $(1, 2)$, and $(2, 1)$, since each lies at a distance of at most 2 from $(1, 0)$, giving a view of beauty $1 + 1 + 1 = 3$. Shooting from $c = 3$ captures the stars at $(2, 1)$ and $(3, 2)$, giving a view of beauty $1 + 1 = 2$. The total beauty is $3 + 2 = 5$.



In the second test case, one optimal choice is $r = 3$, which allows $\lfloor \frac{6}{3} \rfloor = 2$ shots since $k = 6$.

Shooting from $c = 2$ and from $c = 3$ both capture all three stars, since every star lies at a distance of at most 3 from each of these two points. Each of the two views has beauty $5 + 3 + 7 = 15$, so the total beauty is $15 + 15 = 30$.

Problem F. Fraction Again!

Input file:	`standard input`
Output file:	`standard output`
Time limit:	1 second
Memory limit:	256 megabytes

You are given a positive integer k. Find any three positive integers x, y, and z such that

$$\frac{1}{x} + \frac{1}{y} + \frac{1}{z} = k,$$

or report that no such triple exists.

Input

The first line of the input contains a single integer t ($1 \le t \le 100$) — the number of test cases.

The first and only line of each test case contains a single integer k ($1 \le k \le 100$) — the target value of the sum.

Output

For each test case, if a valid triple exists, output three space-separated positive integers x, y, and z on a single line. If there are multiple valid triples, you may output any of them.

If no valid triple exists, output -1 instead.

Example

standard input	standard output
2 1 17	2 3 6 -1

Note

In the first test case, $k = 1$, and $(x, y, z) = (2, 3, 6)$ works because $\frac{1}{2} + \frac{1}{3} + \frac{1}{6} = 1$. Other triples are valid too, for example $(6, 2, 3)$ and $(3, 3, 3)$.

In the second test case, $k = 17$, and it can be shown that no triple of positive integers satisfies the equation, so the answer is -1.

Problem G. Sandfall

Input file:	`standard input`
Output file:	`standard output`
Time limit:	3 seconds
Memory limit:	512 megabytes

You are given a zero-indexed array $a_0, a_1, \ldots, a_{n-1}$ of n integers, each initially equal to 0. You have to perform q update operations on it. Each operation is described by four integers l, r, x, and v, and is performed as follows: for every i with $l \le i \le r$, add v to $a_{x \,\&\, i}$, where & denotes the bitwise AND operation.

Print the array a after all q operations are performed.

Input

The first line contains a single integer t ($1 \le t \le 10^4$) — the number of test cases.

The first line of each test case contains two integers n and q ($1 \le n, q \le 10^6$) — the size of the array and the number of operations.

Each of the next q lines contains four integers l, r, x, and v ($0 \le l \le r \le n-1$; $0 \le x \le n-1$; $1 \le v \le 10^6$) — the parameters of one operation.

It is guaranteed that the sum of n over all test cases does not exceed 10^6 and the sum of q over all test cases does not exceed 10^6.

Output

For each test case, print n integers $a_0, a_1, \ldots, a_{n-1}$ — the final array.

Example

standard input	standard output
3 4 2 1 3 2 5 0 2 3 1 2 1 0 1 1 1000000 5 3 0 4 3 10 2 3 1 6 4 4 2 8	6 1 11 0 1000000 1000000 34 16 10 10 0

Note

In the first test case, the array is initially $[0, 0, 0, 0]$.

The first operation has $l = 1$, $r = 3$, $x = 2$, $v = 5$. For $i = 1$: $2 \,\&\, 1 = 0$, so 5 is added to a_0. For $i = 2$: $2 \,\&\, 2 = 2$, so 5 is added to a_2. For $i = 3$: $2 \,\&\, 3 = 2$, so 5 is added to a_2 again. Note that a_2 receives $v = 5$ twice during this single operation, once for $i = 2$ and once for $i = 3$. The array becomes $[5, 0, 10, 0]$.

The second operation has $l = 0$, $r = 2$, $x = 3$, $v = 1$. Here $3 \,\&\, 0 = 0$, $3 \,\&\, 1 = 1$, and $3 \,\&\, 2 = 2$, so 1 is added to each of a_0, a_1, and a_2. The final array is $[6, 1, 11, 0]$.

Problem H. Hilly Array

Input file:	`standard input`
Output file:	`standard output`
Time limit:	1 second
Memory limit:	512 megabytes

An array h of length l is a *hilly array* if $h_i = \min(i,\ l-i+1)$ for every $1 \le i \le l$. That is, it rises $1, 2, 3, \ldots$ up to a peak of $\lceil l/2 \rceil$ in the middle and then mirrors back down. For example, the hilly array of length 5 is $[1, 2, 3, 2, 1]$ and the one of length 6 is $[1, 2, 3, 3, 2, 1]$. Each length gives exactly one hilly array.

Consider n intervals, where the i-th interval is $[a_i, b_i]$. An interval *contains* a value v if $a_i \le v \le b_i$; if $a_i > b_i$, the interval is empty and contains nothing.

A hilly array of length x *fits* a sequence of intervals if we can find a subsequence of x intervals such that each element of the hilly array, in order, comes from the corresponding interval in the subsequence. Formally, a hilly array h of length x fits a sequence of n intervals if there are positions $1 \le i_1 < i_2 < \cdots < i_x \le n$ such that interval i_j contains h_j for every $1 \le j \le x$. Let f be the length of the longest hilly array that fits, or 0 if none does.

For example, take the 5 intervals $[1, 1], [2, 1], [2, 2], [1, 1], [1, 2]$. The longest hilly array that fits is $[1, 2, 1]$, placed at intervals 1, 3, and 5, which contain 1, 2, and 1 respectively. So $f = 3$ here.

You are given two integers n and k. Compute the sum of f over all k^{2n} ways to form the sequence of intervals, where each a_i and each b_i independently takes a value from 1 to k. Since the answer can be large, output it modulo 998244353.

Input

The first line contains a single integer t ($1 \le t \le 1000$) — the number of test cases.

The first and only line of each test case contains two integers n and k ($1 \le n, k \le 5000$) — the number of intervals and the largest value each a_i and b_i can take.

It is guaranteed that the sum of n over all test cases does not exceed 5000, and the sum of k over all test cases does not exceed 5000.

Output

For each test case, print a single integer — the required sum modulo 998244353.

Example

standard input	standard output
3 2 2 3 3 2 1	16 738 2

Note

In the first test case $n = 2$ and $k = 2$, so each interval is one of $[1, 1]$, $[1, 2]$, $[2, 1]$, $[2, 2]$ (note $[2, 1]$ is empty), giving 16 ways to choose the two intervals. The value of f for each way, with the first interval as the row and the second as the column, is:

	$[1, 1]$	$[1, 2]$	$[2, 1]$	$[2, 2]$
$[1, 1]$	2	2	1	1
$[1, 2]$	2	2	1	1
$[2, 1]$	1	1	0	0
$[2, 2]$	1	1	0	0

For instance:

- Both intervals $[1, 1]$: the length-2 hilly array $[1, 1]$ fits, with a 1 placed in each interval, so $f = 2$.
- First interval $[1, 2]$, second $[2, 2]$: only the first interval contains 1, so $[1]$ fits but $[1, 1]$ does not, and $f = 1$.
- Both intervals $[2, 1]$: both are empty, so no value can be placed anywhere and $f = 0$.

Summing the values in the table gives $2 \cdot 4 + 1 \cdot 8 + 0 \cdot 4 = 16$.

Problem I. Not Today

Input file:	`standard input`
Output file:	`standard output`
Time limit:	1 second
Memory limit:	256 megabytes

Alice is traveling through an undirected connected graph with n nodes and m edges, where multiple edges may connect the same pair of nodes. She starts at node a and wants to reach node b, while Bob tries to prevent her from reaching node b for as long as possible.

Her journey unfolds over a sequence of days. Each day, the following happens in order:

1. Bob blocks exactly one edge of the graph. The block lasts for the current day only; the next day the edge is usable again.
2. Seeing which edge is blocked, Alice either traverses one unblocked edge to an adjacent node, or stays where she is.

Alice plays to reach b in as few days as possible, and Bob plays to delay her for as many days as possible. Assuming both play optimally, determine the number of days Alice needs to reach b, or report that Bob can prevent her from ever reaching node b.

Input

The first line contains a single integer t ($1 \le t \le 10^5$) — the number of test cases.

The first line of each test case contains four integers n, m, a, and b ($2 \le n \le 2 \cdot 10^5$, $n - 1 \le m \le 5 \cdot 10^5$, $1 \le a, b \le n$, $a \ne b$) — the number of nodes, the number of edges, the starting node, and the destination node.

Each of the next m lines contains two integers u and v ($1 \le u, v \le n$, $u \ne v$) — an undirected edge connecting nodes u and v. It is guaranteed that the graph is connected.

It is guaranteed that the sum of n over all test cases does not exceed $2 \cdot 10^5$, and the sum of m over all test cases does not exceed $5 \cdot 10^5$.

Output

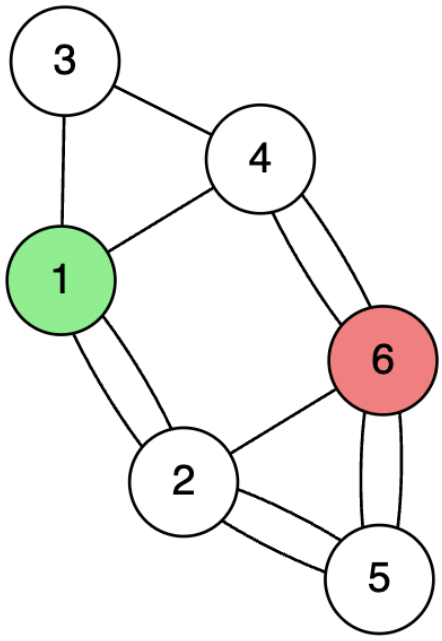
For each test case, print a single line containing the number of days Alice needs to reach b under optimal play, or -1 if Bob can prevent her from ever reaching b.

Example

standard input	standard output
2 6 12 1 6 4 6 5 6 1 3 5 6 2 5 1 2 4 6 2 5 1 4 1 2 6 2 3 4 4 4 1 4 3 2 3 1 2 4 3 4	3 -1

Note

In the first test case, Alice starts from node 1 and reaches node 6 in 3 days. One optimal sequence of moves is shown below.



- On day 1, Bob blocks edge $(1, 4)$, Alice moves to node 2.
- On day 2, Bob blocks edge $(2, 6)$, Alice moves to node 5.
- On day 3, Bob blocks one of the two $(5, 6)$ edges, allowing Alice to use the remaining $(5, 6)$ edge to reach her destination.

In the second test case, Alice can never reach the destination node.

Problem J. Mode Zero

Input file:	`standard input`
Output file:	`standard output`
Time limit:	1 second
Memory limit:	256 megabytes

Consider a binary string w of odd length. As long as its length is at least 3, you may repeatedly apply the following operation:

- Choose any three consecutive characters of w.
- Let x be their *mode*, i.e., the character (`'0'` or `'1'`) that appears at least twice among them.
- Replace these three characters with the single character x.

Formally, one operation on $w = w_1 w_2 \dots w_k$ picks an index i with $1 \le i \le k-2$, computes $x = \text{mode}(w_i, w_{i+1}, w_{i+2})$, and replaces w_i, w_{i+1}, w_{i+2} by the single character x.

We call w *good* if some sequence of operations reduces it to the single character `"0"`.

You are given a binary string s of length n and q queries. Each query is a pair of integers (l, r) with $1 \le l \le r \le n$, where $r - l + 1$ is odd. For each query, determine whether the substring $s[l..r]$ is good. Here, $s[l..r]$ denotes the substring $s_l s_{l+1} \dots s_r$.

Input

The first line of the input contains a single integer t ($1 \le t \le 10^4$) — the number of test cases.

The first line of each test case contains a single integer n ($1 \le n \le 3 \cdot 10^5$) — the length of the binary string.

The second line contains the binary string s of length n.

The third line contains a single integer q ($1 \le q \le 3 \cdot 10^5$) — the number of queries.

Each of the next q lines contains two integers l and r ($1 \le l \le r \le n$; $r - l + 1$ is odd) — a query.

It is guaranteed that the sum of n over all test cases does not exceed $3 \cdot 10^5$, and the sum of q over all test cases does not exceed $3 \cdot 10^5$.

Output

For each query, output `"YES"` if the substring $s[l..r]$ is good, and `"NO"` otherwise.

You can output the answer in any case (upper or lower). For example, the strings `"yEs"`, `"yes"`, `"Yes"`, and `"YES"` will be recognized as positive responses.

Example

standard input	standard output
2 9 100011011 4 1 5 5 9 4 6 3 9 5 00100 2 1 5 3 3	YES NO NO YES YES NO

Note

Consider the first test case, $s = \texttt{100011011}$. In each operation sequence below, the underlined three characters are the ones chosen for that step, and they are replaced by their mode.

The first query is $(1, 5)$, so the substring is 10001. It is good — one possible sequence of operations is:

$$\texttt{10}\underline{\texttt{001}} \rightarrow \underline{\texttt{100}} \rightarrow \texttt{0}$$

The fourth query is $(3, 9)$, so the substring is 0011011. It is good — one possible sequence of operations is:

$$\texttt{00}\underline{\texttt{110}}\texttt{11} \rightarrow \texttt{00}\underline{\texttt{111}} \rightarrow \underline{\texttt{001}} \rightarrow \texttt{0}$$

Problem K. Peak-a-boo

Input file:	`standard input`
Output file:	`standard output`
Time limit:	1 second
Memory limit:	256 megabytes

Let p be an array, initially a permutation* of the integers from 1 to n. An array-processing machine produces a sequence s by repeating the following steps exactly $n-1$ times:

1. Identify all **peaks** of the current array p. An element p_i (where $1 \le i \le |p|$) is a peak if it is strictly greater than its existing adjacent elements; formally, $p_i > p_{i-1}$ (when $i > 1$) and $p_i > p_{i+1}$ (when $i < |p|$).
2. Among all peaks, take the one with the **smallest** value.
3. Append that value to s.
4. Remove that element from p, shifting every later element one position to the left to close the gap.

The process stops when a single element remains in p.

You are given a sequence s of $n-1$ integers. Determine whether some initial permutation of length n produces exactly s. If one exists, construct any such permutation; otherwise, report that it is impossible.

* A permutation of length n is an array consisting of n distinct integers from 1 to n in arbitrary order. For example, $[2,3,1,5,4]$ is a permutation, but $[1,2,2]$ is not a permutation, and $[1,3,4]$ is also not a permutation.

Input

The first line contains a single integer t ($1 \le t \le 10^4$) — the number of test cases.

The first line of each test case contains a single integer n ($2 \le n \le 10^5$) — the length of the initial permutation.

The second line contains $n-1$ integers $s_1, s_2, \ldots, s_{n-1}$ ($1 \le s_i \le n$) — the elements of the sequence s.

It is guaranteed that the sum of n over all test cases does not exceed $2 \cdot 10^5$.

Output

For each test case, print n space-separated integers — any permutation of length n that the machine turns into s. If several such permutations exist, print any of them. If none exists, print `-1`.

Example

standard input	standard output
2 6 3 5 4 6 2 2 1	6 2 5 4 1 3 -1

Note

In the first test case, one valid initial permutation is $[6,2,5,4,1,3]$. The machine processes it as follows, with the removed peak shown in bold:

- $[6,2,5,4,1,\mathbf{3}]$ — the smallest peak is 3.

- $[6, 2, \mathbf{5}, 4, 1]$ — the smallest peak is 5.
- $[6, 2, \mathbf{4}, 1]$ — the smallest peak is 4.
- $[\mathbf{6}, 2, 1]$ — the smallest peak is 6.
- $[\mathbf{2}, 1]$ — the smallest peak is 2.

This yields $s = [3, 5, 4, 6, 2]$.

In the second test case, no valid permutation of length 2 can produce $s = [1]$.

Problem L. Terminal

Input file:	`standard input`
Output file:	`standard output`
Time limit:	1 second
Memory limit:	256 megabytes

You are given an array a of n positive integers $a_1, a_2, \ldots, a_n$.

You may repeatedly apply the following operation:

- Select an index i ($1 \le i < |a|$, where $|a|$ denotes the current length of a) such that $a_i + a_{i+1}$ is odd, and delete a_i. As a result, the elements after index i shift one position to the left.

You must keep applying the operation until there is no valid index where the operation can be applied. That is, you must apply the operation until no two adjacent elements in the array sum up to an odd number. The resulting array is called *terminal*. Depending on the order in which you perform the operations, you can end up with different terminal arrays.

Consider every terminal array reachable from a through some sequence of operations. Determine how many distinct lengths these terminal arrays can have.

Input

The first line contains a single integer t ($1 \le t \le 10^4$) — the number of test cases.

The first line of each test case contains a single integer n ($1 \le n \le 3 \cdot 10^5$) — the length of the array.

The second line contains n integers $a_1, a_2, \ldots, a_n$ ($1 \le a_i \le 10^9$) — the elements of the array.

It is guaranteed that the sum of n over all test cases does not exceed $3 \cdot 10^5$.

Output

For each test case, output a single integer — the number of distinct lengths a terminal array reachable from a can have.

Example

standard input	standard output
2 3 3 5 7 4 1 2 3 4	1 2

Note

In the first test case, every adjacent pair sums up to an even number, so no operation can be performed. The array is already terminal, and the only reachable length is 3.

In the second test case, different orders of operations can leave arrays of different lengths. For instance:

- delete $a_1 = 1$ (since $1 + 2 = 3$ is odd) to get $[2, 3, 4]$, then delete $a_1 = 2$ (since $2 + 3 = 5$ is odd) to get $[3, 4]$, then delete $a_1 = 3$ (since $3 + 4 = 7$ is odd) to get $[4]$, which is terminal with length 1;
- delete $a_3 = 3$ (since $3 + 4 = 7$ is odd) to get $[1, 2, 4]$, then delete $a_1 = 1$ (since $1 + 2 = 3$ is odd) to get $[2, 4]$, which is terminal with length 2.

Only lengths 1 and 2 can be reached, so the answer is 2.